

M13 / LEARNER BOOK

Delta Lake and reliable tables

Standalone Zero-to-Job-Ready Data Engineer 2026

11 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE | How to use this module 3

DE13-01 | Data Lake vs Warehouse vs Lakehouse 4

DE13-02 | Why plain files are not tables 8

DE13-03 | Delta transaction log, ACID and table versions 11

DE13-04 | Create, read, update and delete 15

DE13-05 | Schema enforcement and schema evolution 18

DE13-06 | Time travel and version history 21

DE13-07 | MERGE, upserts and idempotent incremental loads 24

DE13-08 | Slowly Changing Dimension Type 2 with Delta 28

DE13-09 | Bronze, Silver and Gold lakehouse layers 31

DE13-10 | Compaction, OPTIMIZE, VACUUM and maintenance boundaries 35

DE13-11 | Integrated lakehouse pipeline and debugging checkpoint 39

START HERE

How to use this module

Open the matching practice folder from the course root: 12_M13/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Beginner-first lakehouse engineering: from files vs tables through ACID/versioned state, DML, schema control, time travel, MERGE/idempotency, SCD2, medallion layers, maintenance and an integrated debugging checkpoint.

Course runtime baseline: Spark 4.2.0 + Delta Lake 4.4.0. Live Delta/PySpark execution remains learner-PC proof unless explicitly recorded.

Complete M12 Spark/PySpark. Use a separate M13 virtual environment so Delta/Spark compatibility is explicit. External visual material is supplementary; required teaching and practice live in this module. Optional visuals without a verified bounded segment are labelled exactly that way. Official documentation + this learner book + supplied practice are the authoritative route.

Keep Docker/Airflow/PostgreSQL closed while running local Spark on an 8 GB machine unless you deliberately need them. The course profile uses local[2] and bounded shuffle partitions for learning, not as production tuning advice.

Practice: 12_M13/practice/ Quick revision:

12_REVISION_AND_TESTS/M13/M13_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M13/PROTECTED_OPEN_AFTER_ATTEMPT/M13_Module_Revision_Test_Answers.pdf

DE13-01

Data Lake vs Warehouse vs Lakehouse

Watch first

[1. Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes](#)

Conceptual transaction-log/lakehouse walkthrough; version-specific commands may be old

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

A lakehouse lesson is useful only if you can explain the problem it is trying to solve. A data lake gives inexpensive, flexible file storage, but files alone do not automatically provide table-level transactions, dependable updates, or a trustworthy current snapshot. A warehouse gives strong table semantics and SQL-oriented management, but traditionally asks you to load data into a managed analytical system. A lakehouse combines lake-style object/file storage with a table format and transaction layer so engineering teams can keep open data files while gaining reliable table behavior. The point is not that one architecture wins every workload. The point is to choose the guarantees you need and know which layer supplies them.

Picture it in your head

Picture three rooms. The first is a large storeroom where boxes can be placed cheaply: that is the lake. The second is a carefully managed shop floor with labelled shelves, transactions, and controlled access: that is the warehouse. The lakehouse keeps the large storeroom but adds a reliable inventory ledger and table rules so readers can ask, 'What is the committed table right now?' without scanning whatever files happen to exist. Delta Lake is one implementation of that table layer on top of lake storage.

Words before we use them

New word	Meaning before we use it
Data lake	Object/file storage used for raw and processed data; flexibility is high, table guarantees depend on added systems.
Data warehouse	Managed analytical database/platform with table semantics, SQL, governance and optimized serving.
Lakehouse	Architecture that brings table/transaction capabilities to lake storage.
Table format	Metadata + rules that define which data files belong to a table snapshot and how changes are committed.
ACID	Atomicity, consistency, isolation and durability - properties used to reason about reliable transactions.

Useful visual / current reference

Optional video

Official: Delta Lake documentation Lakehouse purpose, ACID transactions, metadata, streaming/batch unification

OPEN SOURCE - CLICK HERE <https://docs.delta.io/>

Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes Conceptual transaction-log/lakehouse walkthrough; version-specific commands may be old

OPEN SOURCE - CLICK HERE <https://www.youtube.com/watch?v=u1VfOiHVeMI>

Follow with me once

Take the same business problem - daily order files plus late corrections - and design it three ways. In a plain file lake, write down how a reader decides which files are current after a correction. In a warehouse, write down where the corrected row is stored and what transaction semantics the database supplies. In a lakehouse, keep the files in lake storage but let the table log decide the committed snapshot. Do not use product names first; name the guarantee first.

```
# Reasoning exercise - no Delta runtime needed
for design in ['lake', 'warehouse', 'lakehouse']:
    print(design, '-> storage / table-state contract / correction rule')
```

What you should see

You should be able to produce a three-column design note that separates storage from table semantics. The lake column should not falsely promise updates or transactions just because the files are Parquet. The warehouse column should acknowledge managed table behavior. The lakehouse column should identify a table layer that tracks committed versions over lake files.

Why it worked

Example: orders arrive as Parquet each hour. At 14:00 an upstream correction changes order O17. If consumers scan every file in a folder, they may see both old and corrected rows unless another convention tells them which one wins. A Delta table records a new committed version whose snapshot includes the files/actions representing the correction. Readers query the table snapshot, not an accidental folder listing.

Break it / common failure

A common beginner failure is to call any collection of Parquet files a lakehouse. Parquet is a columnar file format; it does not by itself provide a transaction log or table history. Another failure is saying a lakehouse 'replaces warehouses.' Real architectures may use both, and the deciding factors include workload, governance, latency, team skills, interoperability and cost.

Reason through a realistic incident

A reporting team says yesterday's revenue changed even though no report code changed. Storage contains the original order files plus a late correction file. Ask: what is the table contract? Can you identify one committed snapshot? Is duplicate/correction precedence encoded in a log/table operation, or is every reader inventing its own rule? The incident is not solved by buying faster storage; it is solved by clarifying state and commit semantics.

Deepen the engineering reasoning

Separate three questions in architecture reviews: where bytes live, how a table snapshot is defined, and which compute engine reads/writes it. Lakehouse thinking becomes clearer when those layers are not collapsed into one vendor name. In this module, Spark is the compute engine, Parquet remains the underlying data-file format, and Delta adds table metadata/transaction semantics. That layered mental model will make later Iceberg/Hudi comparisons easier without needing to learn them now.

Try it yourself

Choose a fictitious logistics company receiving shipment events, corrections and daily aggregates. Write one paragraph for lake, warehouse and lakehouse designs. For each, state where bytes live, who decides the current truth, how a correction is applied, and what a reader must trust.

Fresh changed retry

Now change the scenario: the company has immutable compliance archives that are never updated. Explain why a simple lake zone may be completely appropriate for that zone even if the curated serving layer uses Delta tables.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

Lake, warehouse and lakehouse are architecture choices, not maturity ranks. A lakehouse keeps lake storage but adds table-level metadata and transaction behavior. Always name the required guarantee - current snapshot, update/delete, history, concurrency, schema control - before naming the technology.

Evidence to keep

Save the three-design comparison, the changed compliance scenario, and a 60-90 second explain-back: 'Parquet files are not automatically a transactional table.' Mastery means you can state which layer supplies each guarantee without marketing language.

DE13-02

Why plain files are not tables

Watch first

[1. Optional visual - no verified bounded range: Databricks - Unpacking the Transaction Log](#)

Deep conceptual visual of the log; use for mental model, not version-specific setup

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

M11 taught you file formats and M12 taught you distributed processing. Now the key transition is to stop treating a directory as if it were a database table. A directory can contain valid Parquet files and still have no atomic definition of 'the table at version 7.' Concurrent writers can leave multiple candidate file sets, retries can duplicate output, and readers can observe a mix of old and new files if the application has no commit protocol. Delta's table log turns file actions into a versioned snapshot contract.

Picture it in your head

Imagine a restaurant kitchen receiving paper tickets. The food containers are data files. If tickets are scattered on the floor, you can see containers but you do not know which orders were cancelled, replaced or committed. The transaction log is the ordered ticket book. The current table is reconstructed from committed add/remove actions, not from guessing based on filenames or modification times.

Words before we use them

New word	Meaning before we use it
Snapshot	The set of table metadata and data files that represent one committed version.
Commit	An atomic recorded table change that creates a new version if it succeeds.
AddFile / RemoveFile	Conceptual log actions that add a data file to or remove it from the current snapshot.
<code>_delta_log</code>	Directory containing Delta transaction-log information for a table.
Tombstone	Metadata indicating a file has been logically removed from the current table but may still exist physically until retention cleanup.

Useful visual / current reference

[Optional video](#)

Official: Delta Lake Quick Start Create/read a Delta table and inspect versioned behavior

[OPEN SOURCE - CLICK HERE https://docs.delta.io/quick-start/](https://docs.delta.io/quick-start/)

Optional visual - no verified bounded range: Databricks - Unpacking the Transaction Log Deep conceptual visual of the log; use for mental model, not version-specific setup

[OPEN SOURCE - CLICK HERE https://www.youtube.com/watch?v=IXIBjdehsdw](https://www.youtube.com/watch?v=IXIBjdehsdw)

Follow with me once

Use `practice/delta/01_files_vs_tables.py` on the learner PC. The script writes the same tiny dataset once as Parquet and once as Delta. Inspect the directory names. Do not edit `_delta_log`. Then append one row to the Delta table and inspect history. Your goal is not to memorize filenames; it is to connect a logical table version to recorded commit metadata.

```
parquet_path = BASE / 'parquet_orders'
delta_path = BASE / 'delta_orders'
df.write.mode('overwrite').parquet(str(parquet_path))
df.write.format('delta').mode('overwrite').save(str(delta_path))
print(list(delta_path.iterdir())) # includes _delta_log
```

What you should see

The Parquet directory should contain data files and Spark metadata such as success markers depending on the writer. The Delta directory should contain Parquet data files plus `_delta_log`. After

the append, the Delta table should have another history version. Reading the table through `format('delta')` should return the committed logical rows rather than forcing your script to decide which physical files are active.

Why it worked

Suppose version 0 adds files A and B. A correction creates version 1 that adds file C and logically removes A. The current snapshot is B + C, even if A still exists on disk for time-travel/retention reasons. A naive recursive Parquet reader might read A + B + C and produce wrong results. A Delta reader consults the table log and uses the active snapshot.

Break it / common failure

Do not delete files manually because they 'look old.' Physical presence is not the same as logical membership. Do not use filesystem modified time as a business version. Do not write unrelated files inside a table directory and assume Delta will protect them from every maintenance operation. Table-managed directories need a clear ownership boundary.

Reason through a realistic incident

A junior engineer compares storage size with current table size and says Delta duplicated data. History shows an overwrite and the old files still exist. Diagnose correctly: logical removal can precede physical cleanup so time travel and concurrent readers remain safe. Confirm current row/key totals through the Delta snapshot before discussing retention or VACUUM.

Deepen the engineering reasoning

The important abstraction is not 'Delta has JSON files.' It is that readers and writers agree on a protocol for constructing snapshots and committing changes. Checkpoints can compact log state for efficient reconstruction; the details evolve across versions, but the contract remains: table state comes from committed metadata, not folder intuition. This is why table formats matter for interoperability - independent engines can implement the protocol rather than reverse-engineer one application's filenames.

Try it yourself

Draw versions 0, 1 and 2 for a table using fictional files A-D. For each version, list add/remove actions and calculate the active file set. Then explain why a raw folder scan at version 2 could be wrong if removed files remain physically present.

Fresh changed retry

Change version 2 so a failed job produced an unreferenced Parquet file E without a successful commit. Explain why 'file exists' is not enough to make E part of the table and why manual cleanup must still respect retention/ownership rules.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

A file is a storage object; a table is a logical contract. Delta's transaction log records committed actions and lets readers construct a snapshot. Current table membership is not determined by 'every Parquet file in the directory.'

Evidence to keep

Save the directory inspection, DESCRIBE HISTORY/history output, active-file paper exercise and changed retry. If Delta did not run on your PC, mark runtime PENDING and use the supplied simulator only as reasoning evidence.

DE13-03

Delta transaction log, ACID and table versions

Watch first

[1. Optional visual - no verified bounded range: Databricks - Unpacking the Transaction Log](#)

Conceptual transaction-log walkthrough

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

ACID becomes useful when you connect each letter to a failure you have already seen. Atomicity means a multi-file table change is committed as one logical operation or not exposed as a partial table version. Consistency means declared table invariants are preserved across committed states. Isolation means concurrent readers/writers reason about stable snapshots and conflicts rather than half-written folder contents. Durability means a successful commit remains part of the table history under the storage system's durability guarantees. Delta implements these ideas through an ordered transaction log and versioned snapshots.

Picture it in your head

Think of a bank ledger. A transfer touches multiple balances, but customers should never observe a state where money left account A and has not yet arrived in B. Delta's exact mechanics differ from a bank database, but the mental picture is the same: the commit boundary defines what other readers can consider real. Each successful table commit advances the version.

Words before we use them

New word	Meaning before we use it
Atomicity	Expose all parts of a committed table change together, not a partial result.
Consistency	Committed states respect table/protocol constraints and application invariants.
Isolation	Concurrent operations see/produce controlled snapshots; conflicts are detected according to the protocol.
Durability	Once committed, state survives normal process failure subject to underlying storage durability.
Version	Monotonically advancing committed table state used for history and time travel.
Optimistic concurrency	Writers proceed using a snapshot and validate conflicts when attempting to commit, rather than locking the whole table for the entire job.

Useful visual / current reference

[Optional video](#)

Official: Delta Lake documentation ACID and transactional table guarantees

[OPEN SOURCE - CLICK HERE https://docs.delta.io/](https://docs.delta.io/)

Official: Delta transaction log protocol Protocol-level reference when you want deeper mechanics

[OPEN SOURCE - CLICK HERE https://github.com/delta-io/delta/blob/master/PROTOCOL.md](https://github.com/delta-io/delta/blob/master/PROTOCOL.md)

Optional visual - no verified bounded range: Databricks - Unpacking the Transaction Log
Conceptual transaction-log walkthrough

[OPEN SOURCE - CLICK HERE https://www.youtube.com/watch?v=IXIBjdehsdw](https://www.youtube.com/watch?v=IXIBjdehsdw)

Follow with me once

Run `02_transaction_log_versions.py`. It creates a small Delta table, performs an append and a targeted update, then prints history. Before running, predict how many committed versions you expect. After running, record operation names and verify the latest row/key totals. Do not infer version numbers from how many Parquet files were written - one table commit may add/remove several files.

```
delta = DeltaTable.forPath(spark, str(TABLE))
for row in delta.history().select('version','timestamp','operation').collect():
    print(row)
```

What you should see

You should see sequential history versions for the completed operations. The latest snapshot should reflect the updated business row exactly once. The number of physical files may differ from the number of table versions, because versions represent commits, not files.

Why it worked

Suppose two writers both read version 5. Writer A updates partition/date X and commits version 6. Writer B then tries to commit based on its earlier snapshot. Whether B can safely commit depends on the actions/conflicts involved; the system must not silently combine incompatible changes into an unexplained partial state. Your debugging evidence is table history and conflict/error information, not 'both jobs finished their Python loops.'

Break it / common failure

Do not claim ACID means no job can fail. Jobs fail; ACID helps define what becomes committed and visible. Do not claim isolation means every writer is globally serialized from start to finish. Delta uses optimistic concurrency patterns. Do not equate schema enforcement with every business-quality rule - a string can be valid by schema and still contain an invalid customer code.

Reason through a realistic incident

A pipeline wrote several Parquet files and then crashed before its intended final state. A raw file consumer sees some new files and some old ones. A Delta consumer still reads the latest committed snapshot. Investigate whether a commit was recorded, what version became current, and whether the crashed job should retry idempotently. The right question is not 'did some files get created?' but 'what table version committed?'

Deepen the engineering reasoning

Table history is operational evidence. It can answer which operation created a state, when, and often metrics such as rows/files affected. That does not replace application-level audit columns or source reconciliation, but it gives you a reliable table-state timeline. In production incident response, combine three layers: source evidence, pipeline run evidence and table commit/history evidence. A clean Delta history cannot prove the source delivered the right business data.

Try it yourself

Write a failure matrix for four cases: crash before commit, successful append, conflicting concurrent update, and successful overwrite. For each, state what a reader should consider current and which evidence you would inspect first.

Fresh changed retry

Change the scenario so two jobs touch disjoint date partitions. Explain why 'concurrent' does not automatically mean 'conflict,' but why you should still rely on actual commit behavior and not hard-code assumptions about every operation.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

Delta versions represent committed table states. ACID is about transaction visibility and correctness, not immunity from failure. Debug from source -> job -> commit/history -> reconciled business output.

Evidence to keep

Save the version prediction, actual history, row/key totals, failure matrix and changed retry. Be able to explain why file count and version count are different concepts.

DE13-04

Create, read, update and delete

Watch first

[1. Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes](#)

Conceptual Delta/lakehouse visual; learner book + official docs remain current syntax/runtime authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

A table format earns its place when common data-engineering changes can be expressed as table operations instead of bespoke file-rewrite conventions. Delta supports creation, append/overwrite, reads and DML-style update/delete operations. These operations are powerful because they create new committed snapshots while retaining old files as needed for history/retention. Your responsibility is to use precise predicates, reconcile the result, and understand that logical deletion is not immediate secure erasure of every historical byte.

Picture it in your head

Think of editing a version-controlled spreadsheet where each accepted edit creates a new revision. UPDATE changes matching records in the new revision. DELETE removes matching records from the new logical view. The old revision may remain retrievable until retention cleanup. You should picture a new table snapshot, not an in-place mutation of one Parquet row inside a file.

Words before we use them

New word	Meaning before we use it
DML	Data manipulation operations such as insert/update/delete/merge.
Predicate	Boolean condition selecting rows to change.
Logical delete	Row is removed from current table state; historical files may still exist until cleanup.
Rewrite	Columnar files containing changed rows may be rewritten because Parquet files are immutable objects.
Operation metrics	History fields that can help validate how many rows/files an operation affected.

Useful visual / current reference

[Optional video](#)

Official: Delta table deletes, updates and merges Current semantics and APIs for UPDATE/DELETE/MERGE

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-update/](https://docs.delta.io/delta-update/)

Official: Delta Quick Start Create/read/write/update patterns

[OPEN SOURCE - CLICK HERE https://docs.delta.io/quick-start/](https://docs.delta.io/quick-start/)

Follow with me once

Run `03_crud.py`. It creates four orders, updates one status with a precise `order_id` predicate, deletes one cancelled training row, then prints sorted output and history. Before each DML action, calculate the expected row count, key set and amount sum. After each action, reconcile all three. A successful API call is not your validation.

```
delta.update(condition="order_id = '02'", set={'status': "'closed'"})
delta.delete(condition="order_id = '04' AND status = 'cancelled'")
```

What you should see

The update should change only the intended order's status without changing row count or key set. The delete should reduce row count by exactly one and remove exactly the expected key. History should show the table operations. Your saved controls should make an accidental broad predicate obvious.

Why it worked

If `UPDATE orders SET status='closed' WHERE order_id='02'` targets one known key, you can prove scope before and after. If instead a developer writes `WHERE status IS NOT NULL`, the command may be syntactically valid and catastrophically broad. Engineering safety comes from previewing the predicate population and storing reconciliation controls, not from trusting the operation name.

Break it / common failure

Never demonstrate `UPDATE/DELETE` on valuable data without a safe sandbox, predicate preview and recovery plan. Do not tell a privacy stakeholder that `DELETE` immediately erases historical storage; `VACUUM`/retention and governance policies matter. Avoid using overwrite as a casual substitute for a targeted correction when it increases blast radius.

Reason through a realistic incident

A delete job intended to remove 120 test records reports 1.8 million rows affected. Stop before maintenance cleanup. Preserve history, inspect the predicate and current/previous versions, and determine whether a restore or corrected write is appropriate. Do not run `VACUUM` during investigation because you may destroy the historical files needed for recovery.

Deepen the engineering reasoning

DML correctness has two dimensions: selection correctness and resulting-state correctness. Selection correctness asks whether the predicate matched the intended population. Resulting-state correctness asks whether keys, grain, totals and downstream constraints remain valid after the new commit. Production runbooks should capture both, plus a rollback/restore decision path. Delta gives you table mechanics; it does not know your business invariant unless you encode/check it.

Try it yourself

Extend the fixture with five shipments. Write one safe update and one safe delete. Before executing, print the candidate rows. After executing, reconcile count, key set and a numeric total. Save evidence showing the predicate could not affect unrelated rows.

Fresh changed retry

Change the delete requirement from a specific key to 'all records older than a cutoff except legally retained customers.' Design the predicate preview and validation checks you would require before allowing execution.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

Create/read/update/delete are table operations that produce committed snapshots. Precision comes from predicate preview, row/key/amount reconciliation and history evidence. Logical deletion and physical retention are different concerns.

Evidence to keep

Save candidate-population previews, before/after controls, history rows and the changed legal-retention design. Explain how you would recover from an overly broad DML operation before retention cleanup.

DE13-05

Schema enforcement and schema evolution

Watch first

[1. Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes](#)

Conceptual Delta/lakehouse visual; learner book + official docs remain current syntax/runtime authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Pipelines break when a producer changes structure without a contract. Delta schema enforcement prevents many incompatible writes from silently changing a table. Schema evolution gives you controlled ways to accept intentional structural change. The engineering skill is not 'turn mergeSchema on.' It is deciding whether a change is compatible with the table's meaning, validating downstream consumers, and recording the decision. Automatic evolution is a tool, not permission for producers to mutate contracts freely.

Picture it in your head

Imagine a customs form with required fields. Enforcement is the officer rejecting a shipment whose paperwork does not match the declared form. Evolution is formally updating the form when the business adds a legitimate field. Leaving the gate permanently open because 'new columns might appear' is not governance.

Words before we use them

New word	Meaning before we use it
Schema enforcement	Reject/guard writes that do not match the target table structure according to Delta/Spark rules.
Schema evolution	Intentional change to the target schema, such as adding columns through supported mechanisms.
mergeSchema	Write option that can merge compatible incoming schema changes.
overwriteSchema	Option used in overwrite scenarios when replacing a schema intentionally; high blast radius.
Contract	Agreed structure and semantics between producers and consumers; broader than data types alone.

Useful visual / current reference

[Optional video](#)

[Contents](#)

Official: Delta batch reads and writes Schema validation and schema update/evolution behavior

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-batch/](https://docs.delta.io/delta-batch/)

Official: Delta Lake documentation Schema enforcement as a core Delta capability

[OPEN SOURCE - CLICK HERE https://docs.delta.io/](https://docs.delta.io/)

Follow with me once

Run `04_schema_evolution.py`. First write a baseline customer table. Next attempt to append a DataFrame containing a genuinely new column without enabling schema evolution and save the expected failure. Then repeat using an explicit evolution option and inspect the resulting schema. Finally reconcile old rows: the new column may be null for historical records unless you backfill it.

```
# First attempt should fail for an unexpected new column.
incoming.write.format('delta').mode('append').save(str(TABLE))
# After explicit approval:
incoming.write.format('delta').option('mergeSchema','true').mode('append').save(str(TABLE))
```

What you should see

The uncontrolled append should fail rather than silently changing the target. The controlled evolution should add the new column. Historical rows should remain present and keys unchanged; the new field

should have a predictable value/null pattern. Record the exact schema before and after.

Why it worked

Adding nullable `loyalty_tier` can be a compatible business change if downstream consumers tolerate it. Changing `customer_id` from a stable string identifier to an integer code is not 'just schema evolution'; it may change key semantics. A type-compatible technical write can still violate the business contract, so schema review must include meaning and downstream dependencies.

Break it / common failure

Do not globally enable auto-merge merely to eliminate schema errors. That can turn a useful failure signal into silent drift. Do not use `overwriteSchema` as a first response to one bad batch. Do not treat column presence as proof of valid data: a newly added field may be 100% null or populated with malformed values.

Reason through a realistic incident

A producer deploys a new `discount_code` field Friday evening. The Silver table starts failing. Your options include reject/quarantine until the contract is approved, evolve the schema with explicit consumer checks, or route the field only in Bronze while curated layers remain stable. The wrong response is to bypass enforcement without understanding which downstream jobs select *, serialize fixed schemas or calculate hashes from all columns.

Deepen the engineering reasoning

Schema governance should distinguish additive, breaking and semantic changes. Additive does not automatically mean harmless: column order assumptions, CSV exports, downstream strict schemas and wildcard selects can still break. For important tables, store schema-change evidence alongside deployment/runbook notes. Delta records table metadata changes, but team process decides whether the change was authorized.

Try it yourself

Design an evolution request for an orders table adding `source_channel`. Specify owner, nullable/default behavior, Bronze/Silver/Gold impact, backfill decision, validation query and rollback plan. Then implement the change in the local training table.

Fresh changed retry

Change the request to rename a business-critical column used by dashboards. Explain why this is a different risk class from adding a nullable field and how you would stage compatibility rather than flipping it instantly.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

Enforcement makes drift visible; evolution makes approved change possible. Never confuse 'supported by the engine' with 'safe for the business contract.' Validate schema, keys, null behavior

and consumers.

Evidence to keep

Save the intentional failed write, before/after schemas, key reconciliation, evolution request and changed rename plan. Mastery means you can defend when to reject, evolve or quarantine.

DE13-06

Time travel and version history

Watch first

[1. Optional visual - no verified bounded range: Databricks - Unpacking the Transaction Log](#)

Conceptual Delta transaction-log/version-state visual; learner book + official docs remain authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

When a result changes, engineers need to reproduce what a table looked like at a known point. Delta time travel lets you read previous snapshots by version or timestamp while retained data/log history remains available. This supports debugging, audit investigation and reproducible backfills. It is not an infinite backup system: retention and VACUUM can remove files needed for older versions. Version history is evidence with a lifetime.

Picture it in your head

Picture a stack of transparent maps. Version 0 shows the first road network. Version 1 adds a bridge. Version 2 closes a road. Time travel chooses one map layer without pretending the newer layers never happened. VACUUM can eventually discard old physical pieces, so an old map may no longer be reconstructable forever.

Words before we use them

New word	Meaning before we use it
Time travel	Read a previous table snapshot by version or timestamp.
History	Metadata describing committed table operations/versions.
Retention	Policy controlling how long historical log/data files remain available for reconstruction.
VACUUM	Maintenance operation that removes unreferenced data files older than a retention threshold.
Restore	Create a new commit that makes an earlier table state current again; different from simply reading an old version.

Useful visual / current reference

[Optional video](#)

Official: Delta Quick Start - time travel Reading earlier versions

[OPEN SOURCE - CLICK HERE https://docs.delta.io/quick-start/](https://docs.delta.io/quick-start/)

Official: Delta batch reads/writes - retention and time travel Retention boundaries for historical access

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-batch/](https://docs.delta.io/delta-batch/)

Official: Delta utility commands History and restore operations

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-utility/](https://docs.delta.io/delta-utility/)

Follow with me once

Run `05_time_travel.py`. It creates version 0, appends a change to make version 1, then reads both snapshots and prints history. Write down a control for each version before looking at output. Do not run `VACUUM` as part of this lesson. The objective is to prove reproducibility, not storage cleanup.

```
v0 = spark.read.format('delta').option('versionAsOf', 0).load(str(TABLE))
latest = spark.read.format('delta').load(str(TABLE))
DeltaTable.forPath(spark, str(TABLE)).history().show(truncate=False)
```

What you should see

Version 0 should return the original key set and totals; the latest version should include the appended change. History should show both commits. Your evidence should clearly label which query used an explicit historical version and which used current state.

Why it worked

A finance report generated from table version 42 can be reproduced later by reading version 42 - if its required files/history are still retained. If the team only records 'report ran Tuesday' and uses latest data on rerun, the result may differ because late-arriving corrections created newer versions. Persisting relevant version identifiers in job/report metadata dramatically improves reproducibility.

Break it / common failure

Do not promise time travel forever. Do not run `VACUUM` with aggressively low retention just to save local disk and then expect old versions to work. Do not confuse reading an old version with restoring it: a historical read is non-destructive; `RESTORE` creates a new current commit and can affect downstream incremental consumers.

Reason through a realistic incident

After an accidental delete, an engineer wants to `VACUUM` first because storage looks messy. Stop. Inspect history and verify whether a pre-delete version can still be read. If recovery is required, choose an explicit restore/rewrite path, reconcile the recovered state and only then revisit retention cleanup. Maintenance must not destroy your recovery evidence during the incident.

Deepen the engineering reasoning

Version IDs are better operational anchors than wall-clock guesses because clocks, time zones and delayed commits complicate timestamp reasoning. Still, timestamp travel can be useful when business processes record times rather than versions. For robust pipelines, capture the input table version(s), output version and job/run ID together. That forms a lineage breadcrumb even before a full metadata platform is introduced.

Try it yourself

Create three versions of a small inventory table. For each version, record version ID, row count, distinct key count and quantity sum. Write a one-paragraph incident note explaining how you would reproduce the result of version 1 after version 3 exists.

Fresh changed retry

Assume the table's historical files older than seven days have been vacuumed. Explain what evidence may remain, what data may no longer be readable, and why a separate backup/compliance strategy can still be necessary.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

Time travel reads old snapshots; restore changes current state via a new commit. Historical availability is bounded by retention and physical files. Record versions as part of reproducible pipeline evidence.

Evidence to keep

Save version-specific reconciliations, history output, incident note and retention retry. Be able to distinguish read-old, restore-old and vacuum-old without mixing them.

DE13-07

MERGE, upserts and idempotent incremental loads

Watch first

[1. Optional visual - no verified bounded range: Databricks - Unpacking the Transaction Log](#)

Conceptual Delta transaction-log/version-state visual; learner book + official docs remain authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Incremental pipelines become dangerous when retrying a batch creates duplicates or when late changes overwrite newer facts. MERGE lets you express matched/not-matched behavior against a target table, but MERGE is not automatic idempotency. You still need a stable business key, a deterministic source batch, duplicate-source protection and a rule for whether an incoming change is newer/valid. M08 checkpoint discipline and M09 incremental logic now meet a transactional target.

Picture it in your head

Think of a receptionist comparing an incoming change card with a master register. If the key exists, update only when the card is allowed to supersede the current record. If the key does not exist, insert. If two cards claim the same key with conflicting values, stop and resolve instead of letting processing order choose the winner. Re-running the same accepted cards should leave the register unchanged.

Words before we use them

New word	Meaning before we use it
MERGE	Conditional table operation combining matched and not-matched actions.
Upsert	Update an existing key or insert a new key.
Idempotent	Repeating the same logical input produces the same final business state.
Deduplication boundary	Rule that ensures one authoritative source candidate per target key for a merge.
Watermark/checkpoint	Progress marker used to select incremental source data; it must not outrun durable output.
Late update	Change that arrives after newer batches but may carry an older event/update time.

Useful visual / current reference

[Optional video](#)

Official: Delta deletes, updates and merges MERGE API semantics and examples

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-update/](https://docs.delta.io/delta-update/)

Official: Delta utility history metrics MERGE operation metrics for validation

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-utility/](https://docs.delta.io/delta-utility/)

Follow with me once

Run 06_merge_idempotency.py. The script builds a target orders table and a source batch with one update and one insert. It first validates one source candidate per order_id. The MERGE updates a target row only when the source updated_at is newer, inserts new keys, then reruns the identical batch. Save row count, key count and amount sum after first and second runs; they must match.

```
(target.alias('t').merge(source.alias('s'), 't.order_id = s.order_id')
  .whenMatchedUpdate(condition='s.updated_at > t.updated_at', set={
    'amount': 's.amount', 'updated_at': 's.updated_at'})
  .whenNotMatchedInsertAll().execute())
```

What you should see

After the first run, the existing order should reflect the newer source value and the new order should exist once. After rerun, row count, distinct key count and business totals should be unchanged. History may show an additional MERGE commit even if no business rows change depending on execution behavior, so idempotency is judged by business state, not by 'no new version ever appears.'

Why it worked

Target O2 has updated_at=10:00, amount 80. Source O2 arrives with updated_at=10:05, amount 85, so it may supersede. If a delayed source O2 later arrives with updated_at=09:55, a naive matched-update would roll the table backward. Add a matched condition requiring the source update to be newer. Then reconcile that the older late batch caused no business regression.

Break it / common failure

Do not feed duplicate source keys into MERGE and assume Delta will choose the latest row for you. Ambiguous multiple source matches can fail or create nondeterministic logic depending on the operation/design. Resolve duplicates explicitly before MERGE. Do not advance an upstream checkpoint before the target commit is durable and validated - M08's candidate-checkpoint pattern still applies.

Reason through a realistic incident

A nightly job is retried after the orchestrator times out. The first attempt may actually have committed. The second attempt reprocesses the same source batch. An append-only target doubles revenue; an idempotent MERGE target should converge to the same key state. Investigate source batch identity, target history and reconciliation before deciding whether to skip or rerun.

Deepen the engineering reasoning

Idempotency is end-to-end. A perfect MERGE cannot save a pipeline that reads a moving, nondeterministic source on every retry. Preserve source evidence or a durable batch boundary; validate source key uniqueness; apply deterministic precedence; commit target; reconcile; only then commit external progress. This sequence connects ingestion state, transformation state and table state into one recoverable workflow.

Try it yourself

Add a second source batch containing one newer update, one stale update and one new key. Predict target changes, run the merge, then rerun it. Prove key count and totals are stable on retry and the stale update did not regress the target.

Fresh changed retry

Change the source so two rows share one order_id with different updated_at values. Design a deterministic staging rule that surfaces exact ties as an error rather than arbitrarily ranking them - reuse the ambiguity discipline from M09.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

MERGE expresses conditional target changes; idempotency requires deterministic source selection, stable keys, precedence rules and safe progress commits. Reconcile business state after both first run and retry.

Evidence to keep

Save source duplicate check, first-run controls, retry controls, stale-update proof and duplicate-key retry design. Be ready to explain why 'we use MERGE' is not enough to claim exactly-once business results.

DE13-08

Slowly Changing Dimension Type 2 with Delta

Watch first

[1. Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes](#)

Conceptual Delta/lakehouse visual; learner book + official docs remain current syntax/runtime authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

SCD Type 2 preserves attribute history instead of overwriting it. That matters when analysts need to know what a customer's region, plan or manager was at the time of a fact. The engineering challenge is to expire exactly one current row and insert exactly one new current version for a changed business key, while leaving unchanged keys alone and handling new keys cleanly. Delta MERGE can support this pattern, but the table grain and temporal invariants must be explicit.

Picture it in your head

Imagine an employee file with transparent sheets. When a person changes department, you do not erase the old sheet. You mark its valid-to date and place a new sheet on top with a new valid-from date. At any point there should be one current sheet per employee, and historical sheets must not overlap in contradictory ways.

Words before we use them

New word	Meaning before we use it
Dimension	Descriptive entity table such as customer, product, employee.
SCD Type 2	History-preserving pattern that inserts a new version when tracked attributes change.

Effective-from / effective-to

Validity interval boundaries for each historical version.

Current flag	Convenience marker identifying the active version.
Surrogate key	Warehouse-generated row identifier often used to distinguish historical versions from the stable business key.
Temporal invariant	Rule such as one current row per business key and non-overlapping validity intervals.

Useful visual / current reference

[Optional video](#)

Delta Lake tutorials - SCD Type 2 Curated Delta learning page containing an SCD2 technique reference

[OPEN SOURCE - CLICK HERE https://delta.io/learn/tutorials/](https://delta.io/learn/tutorials/)

Official: Delta MERGE Core operation used to expire/insert versions

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-update/](https://docs.delta.io/delta-update/)

Follow with me once

Run `07_scd2.py`. The fixture starts with current customer versions. The incoming batch contains one unchanged customer, one changed city and one new customer. The script validates one source row per `customer_id`, stages changed rows so the old current version is expired and a new version is inserted, and inserts the new key. Re-run the same batch to prove no second history row is created for an unchanged retry.

```
# Stage changed rows twice: one keyed row expires current target;  
# one null merge_key row inserts the new historical version.  
# Validate exactly one current row per customer_id after commit.
```

What you should see

The changed customer should have two versions: one expired historical row and one current row. The unchanged customer should still have one current row. The new customer should have one current row. A validation query should prove exactly one current row per business key and no duplicate `effective_from` for the same key. Re-running the identical batch should leave these counts unchanged.

Why it worked

Customer C2 changes city from Patna to Pune effective 2026-09-14. Expire the old C2 row by setting `effective_to` to the new effective time and `is_current=false`; insert a new C2 row with city Pune, `effective_from=2026-09-14`, open-ended `effective_to` and `is_current=true`. Facts dated before the change can still join to the historical C2 version using the validity interval.

Break it / common failure

Do not update the old row's attributes in place; that destroys history. Do not insert a new current version without expiring the old one; that produces two current rows. Do not compare every technical column when detecting change - audit timestamps can create a new version every run. Define the tracked business attributes deliberately.

Reason through a realistic incident

A dashboard doubles customer counts after an SCD load. Investigation finds two `is_current=true` rows for some `customer_ids` because the expire step failed but inserts continued. Treat this as a table invariant failure, not a dashboard bug. Quarantine/repair the affected load, prove one current row per key, inspect temporal overlaps, and rerun with transactional/staged logic.

Deepen the engineering reasoning

SCD2 is a model, not a Delta feature. Delta supplies transaction and merge mechanics; you supply dimensional meaning. In a full warehouse, facts may store surrogate keys resolved at load time, or joins may use business key plus event timestamp against validity intervals. Both require clean temporal intervals. Your module goal is not to memorize one clever MERGE statement but to reason from invariants and validate them after every load.

Try it yourself

Track `customer_segment` instead of `city`. Add one changed, one unchanged and one new customer. Implement/load the change, then write validation queries for one-current-row, no duplicate version start, and expected historical version counts.

Fresh changed retry

Now receive two changes for the same customer in the same source batch with different effective times. Explain why the simple one-row-per-key staging contract rejects this batch and what preprocessing/ordering logic would be required before SCD2 application.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

SCD2 preserves descriptive history by expiring old versions and inserting new ones. Delta provides MERGE/transaction mechanics; correctness comes from temporal invariants, deterministic change detection and source-key discipline.

Evidence to keep

Save before/after dimension rows, invariant checks, idempotent retry proof and multi-change reasoning. Explain why one current row per business key is necessary but not sufficient - interval overlap checks matter too.

DE13-09

Bronze, Silver and Gold lakehouse layers

Watch first

[1. Optional visual with publisher chapter markers: Medallion Architecture in Databricks - Bronze, Silver, Gold](#)

00:00-01:36 medallion concept; implementation chapters later in the video. UI/vendor details are supplementary.
Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Medallion layering is useful when each layer has a clear contract, not when teams create three folders because a diagram said so. Bronze preserves source evidence with minimal irreversible transformation. Silver applies validated structure, deduplication, conformance and quality rules. Gold serves business-ready aggregates or dimensional outputs. The exact names can change, but separating replayable source evidence from curated business state makes recovery and ownership clearer.

Picture it in your head

Think of a water-treatment plant. Bronze is the intake reservoir where you preserve what arrived and when. Silver is the treatment stage where contaminants are identified, rules applied and water standardized. Gold is the distribution tank prepared for a specific consumer need. You do not pour Gold back into Bronze or discard intake evidence just because treatment succeeded.

Words before we use them

New word	Meaning before we use it
Bronze	Raw/landing table layer preserving source fidelity and ingestion metadata.
Silver	Validated/conformed layer with explicit quality, typing, dedupe and business-cleaning rules.
Gold	Serving layer optimized for business use cases such as aggregates, marts or dimensional outputs.
Replayability	Ability to rebuild downstream state from preserved upstream evidence.
Quarantine	Explicit place/state for records that fail quality rules without silently disappearing.
Lineage	Trace of which upstream data and transformations produced an output.

Useful visual / current reference

[Optional video](#)

Official: Delta Lake documentation Lakehouse and batch/stream table capabilities

[OPEN SOURCE - CLICK HERE https://docs.delta.io/](https://docs.delta.io/)

Databricks architecture reference - medallion lakehouse Current platform explanation; use as architecture reference, not an OSS Delta requirement

[OPEN SOURCE - CLICK HERE https://docs.databricks.com/aws/en/lakehouse/medallion](https://docs.databricks.com/aws/en/lakehouse/medallion)

Follow with me once

Run 08_medallion.py. It writes a Bronze orders table including ingestion metadata and one deliberately bad row. Silver applies explicit types, key/amount validation and separates invalid records into a quarantine table instead of dropping them silently. Gold aggregates valid Silver rows by business date. Reconcile counts across Bronze = Silver-valid + quarantine, then reconcile Gold amount totals back to Silver.

```
assert bronze_count == silver_count + quarantine_count
assert round(silver_sales, 2) == round(gold_sales, 2)
```

What you should see

Every Bronze row must be accounted for either in valid Silver output or quarantine. Gold totals must equal the corresponding Silver valid totals for the metric definition. The layers should be separate

Delta tables/paths, with Bronze evidence retained. A Gold aggregate should not become the only copy of detailed business facts.

Why it worked

If Bronze has 6 rows, one with negative quantity, Silver may contain 5 valid rows and quarantine 1 rejected row. Gold daily revenue should aggregate the 5 valid rows only. Your control equation is $6 = 5 + 1$, plus $\text{sum}(\text{Silver.gross_sales}) = \text{sum}(\text{Gold.daily_sales})$. That simple reconciliation is more trustworthy than saying 'the notebook ran green.'

Break it / common failure

Do not transform Bronze so heavily that original source evidence is lost. Do not let Silver silently drop rows without a reconciliation path. Do not build Gold directly from raw sources when Silver defines the canonical cleaning/conformance rules unless the exception is deliberate and documented. And do not assume Bronze/Silver/Gold means three copies of every dataset forever; retention and design depend on requirements.

Reason through a realistic incident

A Gold KPI is 4% lower than source finance totals. Start at Gold-to-Silver reconciliation, then Silver-to-Bronze/quarantine, then Bronze-to-source manifest. You may discover an intentional quality rejection, a duplicate rule, a late source batch or a Gold filter. Layering helps because each boundary has a contract and control total; without that, engineers jump randomly across notebooks.

Deepen the engineering reasoning

Medallion design is strongest when paired with idempotent rebuild rules. If Silver can be deterministically rebuilt from immutable/preserved Bronze evidence, incident recovery becomes simpler. Gold can then be rebuilt from stable Silver. This is not always free - large histories cost storage/compute - so production designs balance replay window, retention and recovery objectives. Keep the course pattern simple and explicit before adding streaming/change-data-feed complexity in M14.

Try it yourself

Design Bronze/Silver/Gold for parcel scans. State one-row grain, keys, retained raw fields, Silver quality rules, quarantine behavior, Gold consumer and two reconciliation equations. Then implement one small variant using the practice scaffold.

Fresh changed retry

Now add a GDPR deletion requirement. Explain which layers and historical copies may be affected and why a simple 'delete from Gold' is not a complete governance response.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

Bronze preserves source evidence, Silver creates validated/conformed truth, Gold serves business use. The value is clear contracts and replay/reconciliation, not the medal names themselves.

Evidence to keep

Save layer schemas, Bronze/Silver/quarantine counts, Silver/Gold totals, design note and privacy retry. Be able to trace one Gold number back to source evidence.

DE13-10

Compaction, OPTIMIZE, VACUUM and maintenance boundaries

Watch first

[1. Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes](#)

Conceptual Delta/lakehouse visual; learner book + official docs remain current syntax/runtime authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Reliable tables can still become slow or expensive if every tiny batch creates many small files. Compaction combines smaller files into better-sized files without changing business rows. VACUUM is different: it physically removes unreferenced files beyond retention, reducing storage but potentially eliminating old-version readability. Maintenance must preserve business results and respect recovery/concurrency requirements. Performance work never gets permission to erase evidence casually.

Picture it in your head

Picture a library. Compaction is rebinding many thin pamphlets into fewer well-organized volumes while keeping the same text. VACUUM is throwing away withdrawn old editions after the retention policy says they are no longer needed. One changes layout; the other changes what historical physical material remains available.

Words before we use them

New word	Meaning before we use it
Small-file problem	Too many small data files create metadata/opening overhead and inefficient scans.
Compaction / bin-packing	Rewrite layout into fewer/larger files while preserving table rows.
OPTIMIZE	Delta command/API for file compaction; can return metrics.
VACUUM	Remove unreferenced data files older than retention threshold.
Retention threshold	Minimum age for eligible unreferenced files; must exceed safety/recovery needs.
dataChange	Metadata concept indicating whether a commit represents business data changes versus layout-only actions.

Useful visual / current reference

[Optional video](#)

Official: Delta optimizations OPTIMIZE/bin-packing behavior and idempotency

[OPEN SOURCE - CLICK HERE https://docs.delta.io/optimizations-oss/](https://docs.delta.io/optimizations-oss/)

Official: Delta utility commands VACUUM behavior, retention cautions and history metrics

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-utility/](https://docs.delta.io/delta-utility/)

Official: Delta batch reads/writes Retention properties and time-travel boundary

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-batch/](https://docs.delta.io/delta-batch/)

Follow with me once

Run 09_maintenance.py only in the disposable learner table. The script creates several small append commits, records row/key/amount controls and table detail, runs Delta optimize().executeCompaction(), then reruns the same business controls. It prints the safe VACUUM command as a learning reference but does not execute destructive shortened-retention cleanup. Use default/safe retention in real practice unless you fully understand the consequences.

```
delta = DeltaTable.forPath(spark, str(TABLE))
metrics = delta.optimize().executeCompaction()
metrics.show(truncate=False)
# VACUUM is intentionally not run with unsafe short retention in this course.
```

What you should see

Business row count, distinct keys and totals must be identical before and after compaction. File metrics may improve. A second compaction may do little because bin-packing is designed to be idempotent for unchanged data. No course script should disable retention safety merely to demonstrate deletion.

Why it worked

A table has 2,000 tiny files from micro-batches. Compaction may replace them with fewer larger files in a new table version while keeping the same rows. Old files can remain physically present until VACUUM eligibility. If you immediately measure raw directory size, it may temporarily increase even though current query layout improved. Distinguish current active files from retained historical files.

Break it / common failure

Do not equate OPTIMIZE with removing duplicates; it changes physical layout, not business grain. Do not use VACUUM as a performance fix for bad query plans. Do not disable retention checks or set zero-hour retention in a beginner lab. Do not run maintenance during an unresolved corruption incident when historical versions may be needed for comparison/recovery.

Reason through a realistic incident

Storage cost spikes after a series of overwrites. An engineer proposes aggressive VACUUM. First inspect history, active file metrics, retention policy, longest-running readers/streams and recovery requirements. Maybe old files are legitimately retained; maybe compaction is needed; maybe the pipeline rewrites too much. Choose maintenance only after identifying the cause.

Deepen the engineering reasoning

Maintenance has three separate objectives: read performance/layout, storage cleanup and historical recoverability. They can conflict. A platform may optimize layout frequently but vacuum conservatively. Record maintenance operations in operational evidence and monitor their metrics. In cloud object storage, file count can affect listing/metadata overhead even when total bytes are modest; in local training, we simulate the reasoning at small scale without pretending it benchmarks production.

Try it yourself

Create 10 tiny append commits in a disposable table. Record history count, detail/file metrics and business controls. Compact, then reconcile business state. Write a proposed VACUUM retention policy for a hypothetical daily pipeline with a 3-day maximum outage/recovery window and explain your safety margin.

Fresh changed retry

Change the system so a streaming consumer can lag for five days. Explain how that changes retention thinking and why a generic '7 days is always enough' rule is not engineering reasoning.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

OPTIMIZE changes layout; VACUUM removes eligible unreferenced physical files. Both require before/after evidence, but only VACUUM can destroy old-version readability. Never trade recoverability for a demo shortcut.

Evidence to keep

Save pre/post compaction controls, metrics/history, retention proposal and lag retry. State clearly whether live OPTIMIZE/VACUUM was executed or only reviewed on your machine.

DE13-11

Integrated lakehouse pipeline and debugging checkpoint

Watch first

[1. Optional visual - no verified bounded range: Databricks - How Delta Lake Supercharges Data Lakes](#)

Conceptual Delta/lakehouse visual; learner book + official docs remain current syntax/runtime authority.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

The module is complete only when you can connect the pieces under failure: preserve a replayable source boundary, write Bronze, validate and curate Silver, apply idempotent MERGE/SCD behavior where needed, serve Gold, inspect history, reproduce an older version, and separate layout maintenance from business correction. The integrated checkpoint is deliberately small enough for a laptop but uses the same reasoning order you will need in production.

Picture it in your head

Picture an evidence chain rather than a pile of scripts: source batch -> Bronze commit -> Silver validation/quarantine -> curated Delta commit -> Gold output -> reconciliations -> history/version IDs -> run record. If something breaks, move along the chain and find the first boundary where expected and actual state diverge. Do not start by randomly changing Spark settings.

Words before we use them

New word	Meaning before we use it
Run contract	Documented inputs, batch identity, expected invariants, outputs and completion conditions for one pipeline execution.
Reconciliation	Independent comparison proving populations/totals are preserved or intentionally changed.
Recovery point	Known source/table version from which the pipeline can safely replay/rebuild.
Blast radius	Scope of data/consumers potentially affected by an operation.
Runbook	Operational instructions for executing, validating, diagnosing and recovering a pipeline.

Useful visual / current reference

[Optional video](#)

Official: Delta Lake documentation Core Delta capabilities and conceptual reference

[OPEN SOURCE - CLICK HERE https://docs.delta.io/](https://docs.delta.io/)

Official: Delta utility commands History/detail/restore/vacuum operational tools

[OPEN SOURCE - CLICK HERE https://docs.delta.io/delta-utility/](https://docs.delta.io/delta-utility/)

Delta Lake 4.4.0 release Verified Spark 4.2.0 compatibility baseline for this module

[OPEN SOURCE - CLICK HERE https://github.com/delta-io/delta/releases/tag/v4.4.0](https://github.com/delta-io/delta/releases/tag/v4.4.0)

Apache Spark 4.2.0 release/news Verified Spark 4.2 line baseline

[OPEN SOURCE - CLICK HERE https://spark.apache.org/news/](https://spark.apache.org/news/)

Follow with me once

Use `10_integrated_checkpoint.py` after completing the earlier scripts. It creates a small Bronze source batch, builds Silver with quality controls, applies an idempotent order update into a curated table, creates a Gold aggregate, records table versions and writes a JSON evidence summary. Run

once, save controls, then run again with the same batch and prove the curated business state is unchanged. The script does not advance any external checkpoint until output validation succeeds.

```
evidence = {
    'source_rows': source_count, 'silver_rows': silver_count,
    'quarantine_rows': quarantine_count, 'gold_sales': gold_sales,
    'curated_keys': curated_keys, 'curated_version': latest_version,
}
json.dump(evidence, open(EVIDENCE_FILE, 'w'), indent=2, default=str)
```

What you should see

The evidence summary should contain source/Bronze/Silver/Gold counts, key counts, business totals, quarantine count and relevant Delta history/version information. A second identical run should preserve curated row/key/totals. If live Delta is unavailable, the pure-Python `simulators/lakehouse_state_simulator.py` can verify logic structure, but runtime mastery remains PENDING until the learner PC executes Delta 4.4.0 + Spark 4.2.0 successfully.

Why it worked

Suppose the second run produces one extra curated row. Work backward: Did source batch identity change? Did the staging query duplicate a key? Did MERGE use the stable business key? Did a Bronze append occur twice but Silver dedupe fail? Which reconciliation first diverged? The fastest debugger follows controls, not intuition. Table history tells you what committed; source/run metadata tells you why the job attempted it.

Break it / common failure

Do not call the checkpoint complete because every script exited zero. Deliberately break one case: duplicate source key, invalid amount, stale update or schema drift. Save the first failure, diagnose it, repair one cause, rerun and prove reconciliation. Do not hide the error by `dropDuplicates` without a deterministic rule or by overwriting the whole table.

Reason through a realistic incident

At 08:00 Gold revenue is wrong. Bronze count matches source manifest. Silver valid + quarantine matches Bronze, but Silver revenue is too high. That narrows the failure to Silver transformation/dedupe rather than ingestion. Inspect key uniqueness and merge staging before touching Gold. If the wrong Silver commit is isolated and history retained, use time travel to compare versions and build a corrected forward commit/recovery plan.

Deepen the engineering reasoning

A production lakehouse is an operational system, not a storage slogan. The strongest habit from M13 is to treat every table change as a state transition with evidence: intended input, deterministic transformation, transactional commit, independent reconciliation, history/version capture and safe progress. M14 will add streaming/event-driven behavior, where the same state discipline becomes even more important because retries and continuous processing amplify weak assumptions.

Try it yourself

Run the integrated checkpoint with a changed batch that includes a new key, a newer correction, one invalid row and one unchanged retry. Produce a one-page run report: source batch ID, counts, key controls, totals, quarantine reasons, table versions, one failure/recovery note and final status.

Fresh changed retry

Inject two conflicting source rows for the same key with the same updated_at but different amounts. The correct outcome is to surface ambiguity and stop before target progress. Explain why choosing whichever row appears last would make retries/order changes unsafe.

Validation and recovery ladder

- State the table grain and stable business key before changing data.
- Record before/after row count, distinct key count and at least one business total or invariant.
- Inspect table history/version evidence; do not infer committed state from physical file presence.
- When a source batch can contain multiple rows per target key, make precedence deterministic or stop on ambiguity.
- Repair one cause at a time and rerun a changed case; never hide an unexplained mismatch with overwrite or dropDuplicates.
- Keep protected reviews closed until your genuine attempt and evidence are saved.

Revision in your own words

A trustworthy lakehouse pipeline is a chain of deterministic state transitions with preserved source evidence, transactional table commits, reconciliation and recoverable history. Debug the first boundary where controls diverge.

Evidence to keep

Save the integrated run report, live commands/results, one deliberate failure, recovery proof, idempotent retry and 90-second teach-back. If any runtime step was not executed on your learner PC, label it PENDING instead of converting static/simulator QA into mastery.